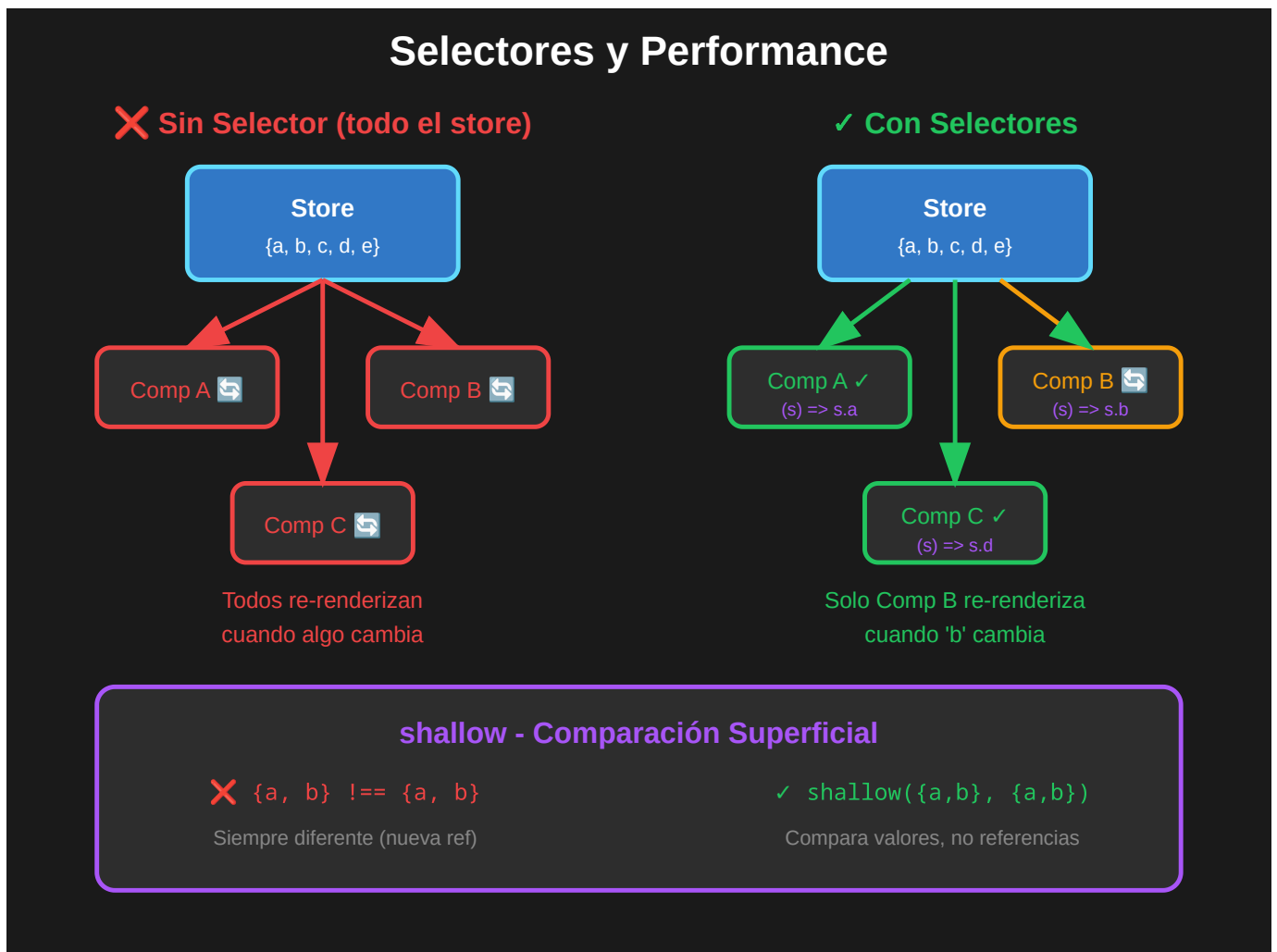


Selectores y Performance

Objetivos

- Entender cómo Zustand maneja re-renders
- Crear selectores eficientes
- Usar shallow para comparación superficial
- Implementar selectores derivados
- Optimizar performance con memoización

1. El Problema de los Re-renders



Sin Selectores: Re-render Excesivo

```
// =====  
// QUÉ: Suscripción completa al store  
// PARA: Obtener todo el estado (anti-patrón)  
// IMPACTO: Re-render en CUALQUIER cambio del store  
// =====  
  
// ✗ MAL: Suscribirse a todo el store  
const BadComponent: React.FC = () => {
```

```
// Se re-renderiza cuando CUALQUIER cosa cambia en el store
const store = useProductStore();

console.log('BadComponent render'); // Muchos renders innecesarios

return <p>Productos: {store.products.length}</p>;
};
```

Con Selectores: Re-renders Optimizados

```
// =====
// QUÉ: Suscripción selectiva a partes del store
// PARA: Re-render solo cuando ESO cambia
// IMPACTO: Mejor performance, menos renders
// =====

// ✅ BIEN: Selector específico
const GoodComponent: React.FC = () => {
  // Solo se re-renderiza cuando products.length cambia
  const productCount = useProductStore((state) => state.products.length);

  console.log('GoodComponent render'); // Solo cuando es necesario

  return <p>Productos: {productCount}</p>;
};
```

2. Tipos de Selectores

Selector Simple

```
// =====
// QUÉ: Selector que extrae un valor directo
// PARA: Acceder a una propiedad del estado
// IMPACTO: Re-render solo si esa propiedad cambia
// =====

const ProductCount: React.FC = () => {
  // Selector simple - extrae products
  const products = useProductStore((state) => state.products);

  return <p>{products.length} productos</p>;
};

const LoadingIndicator: React.FC = () => {
  // Selector simple - extrae isLoading
  const isLoading = useProductStore((state) => state.isLoading);

  if (!isLoading) return null;
  return <div className="spinner">Cargando...</div>;
};
```

Selector Derivado (Computado)

```
// =====
// QUÉ: Selector que calcula un valor derivado
// PARA: Valores que dependen de otros valores
// IMPACTO: Cálculo en cada render, usar con cuidado
// =====

const CartSummary: React.FC = () => {
  // Selector derivado - calcula el total
  const total = useCartStore((state) =>
    state.items.reduce((sum, item) => sum + item.price * item.quantity, 0),
  );

  const itemCount = useCartStore((state) =>
    state.items.reduce((sum, item) => sum + item.quantity, 0),
  );

  return (
    <div>
      <p>{itemCount} artículos</p>
      <p>Total: ${total.toFixed(2)}</p>
    </div>
  );
};
```

Selector con Filtrado

```
// =====
// QUÉ: Selector que filtra datos
// PARA: Obtener subconjuntos de datos
// IMPACTO: Crea nuevo array en cada render
// =====

const CompletedTodos: React.FC = () => {
  // ⚠ Crea nuevo array cada vez - puede causar re-renders
  const completedTodos = useTodoStore((state) =>
    state.todos.filter((todo) => todo.completed),
  );

  return (
    <ul>
      {completedTodos.map((todo) => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
};
```

3. Shallow Equality

El Problema con Objetos/Arrays

```
// =====
// QUÉ: Problema de referencia con objetos
```

```
// PARA: Entender por qué shallow es necesario
// IMPACTO: Sin shallow, siempre re-renderiza
// =====

// ❌ PROBLEMA: Objeto nuevo en cada selector call
const BadMultiSelect: React.FC = () => {
  // Esto crea un NUEVO objeto cada vez
  // {} !== {} siempre es true → re-render infinito
  const { name, email } = useUserStore((state) => ({
    name: state.user?.name,
    email: state.user?.email,
  }));

  return (
    <p>
      {name} - {email}
    </p>
  );
};
```

Solución con shallow

```
// =====
// QUÉ: shallow compara propiedades superficialmente
// PARA: Evitar re-renders cuando los valores son iguales
// IMPACTO: Comparación por valor, no por referencia
// =====

import { shallow } from 'zustand/shallow';

// ✅ SOLUCIÓN: Usar shallow para comparación
const GoodMultiSelect: React.FC = () => {
  const { name, email } = useUserStore(
    (state) => ({
      name: state.user?.name,
      email: state.user?.email,
    }),
    shallow, // Compara { name, email } superficialmente
  );

  return (
    <p>
      {name} - {email}
    </p>
  );
};

// ✅ ALTERNATIVA: Selectores separados (más simple)
const AltMultiSelect: React.FC = () => {
  const name = useUserStore((state) => state.user?.name);
  const email = useUserStore((state) => state.user?.email);

  return (
    <p>
      {name} - {email}
    </p>
  );
};
```

```
    </p>
  );
};
```

Cuándo Usar shallow

```
// =====
// QUÉ: Guía de cuándo usar shallow
// PARA: Decisiones correctas de optimización
// IMPACTO: Código limpio y performante
// =====

// ✅ NO necesitas shallow - valor primitivo
const count = useStore((state) => state.count);

// ✅ NO necesitas shallow - referencia estable (función)
const increment = useStore((state) => state.increment);

// ⚠️ NECESITAS shallow - objeto nuevo
const { a, b } = useStore((state) => ({ a: state.a, b: state.b }), shallow);

// ⚠️ NECESITAS shallow - array nuevo
const items = useStore((state) => state.items.filter((x) => x.active), shallow);

// ✅ ALTERNATIVA a shallow - selectores individuales
const a = useStore((state) => state.a);
const b = useStore((state) => state.b);
```

4. Selectores Memoizados

Con useMemo (en componente)

```
// =====
// QUÉ: Memoizar cálculos derivados costosos
// PARA: Evitar recálculos innecesarios
// IMPACTO: Mejor performance en listas grandes
// =====

import { useMemo } from 'react';
import { shallow } from 'zustand/shallow';

const FilteredProductList: React.FC = () => {
  const { products, filters } = useProductStore(
    (state) => ({
      products: state.products,
      filters: state.filters,
    }),
    shallow,
  );

  // Memoizar el filtrado
  const filteredProducts = useMemo(() => {
    console.log('Calculando productos filtrados...'); // Solo cuando cambia
```

```

return products.filter((product) => {
  if (
    filters.search &&
    !product.name.toLowerCase().includes(filters.search.toLowerCase())
  ) {
    return false;
  }
  if (filters.category && product.category !== filters.category) {
    return false;
  }
  if (filters.minPrice && product.price < filters.minPrice) {
    return false;
  }
  if (filters.maxPrice && product.price > filters.maxPrice) {
    return false;
  }
  return true;
});
}, [products, filters]);

return (
  <ul>
    {filteredProducts.map((product) => (
      <li key={product.id}>{product.name}</li>
    ))}
  </ul>
);
};

```

Selectores en el Store

```

// =====
// QUÉ: Definir selectores como parte del store
// PARA: Reutilizar lógica de selección
// IMPACTO: DRY, lógica centralizada
// =====

interface TodoStore {
  todos: Todo[];
  filter: 'all' | 'active' | 'completed';

  // Acciones
  addTodo: (text: string) => void;
  toggleTodo: (id: number) => void;
  setFilter: (filter: 'all' | 'active' | 'completed') => void;

  // Selectores (getters)
  getFilteredTodos: () => Todo[];
  getStats: () => { total: number; completed: number; active: number };
}

const useTodoStore = create<TodoStore>((set, get) => ({
  todos: [],
  filter: 'all',

```

```

addTodo: (text) =>
  set((state) => ({
    todos: [...state.todos, { id: Date.now(), text, completed: false }],
  })),

toggleTodo: (id) =>
  set((state) => ({
    todos: state.todos.map((t) =>
      t.id === id ? { ...t, completed: !t.completed } : t,
    ),
  })),

setFilter: (filter) => set({ filter }),

// Selector como función del store
getFilteredTodos: () => {
  const { todos, filter } = get();
  switch (filter) {
    case 'active':
      return todos.filter((t) => !t.completed);
    case 'completed':
      return todos.filter((t) => t.completed);
    default:
      return todos;
  }
},

getStats: () => {
  const todos = get().todos;
  const completed = todos.filter((t) => t.completed).length;
  return {
    total: todos.length,
    completed,
    active: todos.length - completed,
  };
},
}));

// Uso en componente
const TodoList: React.FC = () => {
  const getFilteredTodos = useTodoStore((state) => state.getFilteredTodos);
  const filteredTodos = getFilteredTodos();

  return (
    <ul>
      {filteredTodos.map((todo) => (
        <li key={todo.id}>{todo.text}</li>
      ))}
    </ul>
  );
};

```

5. Patrones de Performance

Componentes Separados

```
// =====
// QUÉ: Separar componentes por suscripción
// PARA: Aislar re-renders
// IMPACTO: Solo el componente afectado se re-renderiza
// =====

// ❌ MAL: Un componente con todo
const BadCart: React.FC = () => {
  const items = useCartStore((state) => state.items);
  const total = useCartStore((state) =>
    state.items.reduce((sum, i) => sum + i.price * i.quantity, 0)
  );
  const addItem = useCartStore((state) => state.addItem);

  // Todo se re-renderiza cuando cualquier cosa cambia
  return (
    <div>
      <CartItems items={items} />
      <CartTotal total={total} />
      <AddItemButton onAdd={addItem} />
    </div>
  );
};

// ✅ BIEN: Componentes separados
const CartItems: React.FC = () => {
  const items = useCartStore((state) => state.items);
  return (/* render items */);
};

const CartTotal: React.FC = () => {
  const total = useCartStore((state) =>
    state.items.reduce((sum, i) => sum + i.price * i.quantity, 0)
  );
  return <p>Total: ${total}</p>;
};

const AddItemButton: React.FC = () => {
  const addItem = useCartStore((state) => state.addItem);
  return <button onClick={() => addItem(/* item */)}>Agregar</button>;
};

const GoodCart: React.FC = () => {
  return (
    <div>
      <CartItems />
      <CartTotal />
      <AddItemButton />
    </div>
  );
};
```


useShallow Hook (Zustand v4.4+)

```
// =====  
// QUÉ: Hook useShallow para selección múltiple  
// PARA: Sintaxis más limpia que shallow  
// IMPACTO: Mismo resultado, mejor DX  
// =====  
  
import { useShallow } from 'zustand/react/shallow';  
  
const UserProfile: React.FC = () => {  
  // useShallow es más limpio que el segundo parámetro  
  const { name, email, avatar } = useUserStore(  
    useShallow((state) => ({  
      name: state.name,  
      email: state.email,  
      avatar: state.avatar,  
    })),  
  );  
  
  return (  
    <div>  
      <img  
        src={avatar}  
        alt={name}  
      />  
      <h2>{name}</h2>  
      <p>{email}</p>  
    </div>  
  );  
};
```

6. Debugging de Re-renders

```
// =====  
// QUÉ: Herramientas para debuggear re-renders  
// PARA: Identificar problemas de performance  
// IMPACTO: Optimización informada  
// =====  
  
import { useEffect, useRef } from 'react';  
  
// Hook para contar renders  
const useRenderCount = (componentName: string) => {  
  const renderCount = useRef(0);  
  renderCount.current += 1;  
  
  useEffect(() => {  
    console.log(`${componentName} rendered ${renderCount.current} times`);  
  });  
  
  return renderCount.current;  
};
```

```
// Uso
const MyComponent: React.FC = () => {
  const renderCount = useRenderCount('MyComponent');
  const data = useStore((state) => state.data);

  return (
    <div>
      <p>Renders: {renderCount}</p>
      <p>Data: {data}</p>
    </div>
  );
};
```

Recursos Adicionales

- [Zustand - Selecting State](#)
- [React DevTools Profiler](#)
- [Why Did You Render](#)

Checklist de Comprensión

- ☐ Entiendo cómo los selectores previenen re-renders
- ☐ Sé cuándo usar shallow
- ☐ Puedo crear selectores derivados con useMemo
- ☐ Sé separar componentes para optimizar renders
- ☐ Puedo debuggear problemas de re-renders

Siguiente: [05-persistencia-y-middleware.md](#)